



SOICT

PROJECT REPORT

Weather Data Analysis

Course: IT4043E

Supervisor: Dr. Tran Viet Trung

Authors:

Trieu Kinh Quoc
Vu Quoc Dung
Phung Tien Thanh
Tran Khoi Nguyen
Le Minh Hai

Student ID:

20225524
20225488
20225531
20225453
20225491

Hanoi, December 2025

Table of Contents

1 Problem Definition	3
1.1 Project Introduction	3
1.2 Data Introduction	3
1.3 Problem suitability for big data	4
1.4 Scope and Limitations	4
2 Architecture and Design	5
2.1 Overall architecture	5
2.2 Detail components	5
2.2.1 Kafka component	5
2.2.2 Spark components	6
2.2.2.1 Spark Streaming Job (Speed Layer):	6
2.2.2.2 Spark Batch Job (Batch Layer)	7
2.2.2.3 Extra: Simple Linear Regression with MLlib	8
2.2.3 InfluxDB component	9
2.2.4 Grafana component	10
2.2.5 HDFS components	10
2.2.5.1 NameNode	11
2.2.5.2 DataNode	11
2.2.6 YARN components	11
2.2.6.1 ResourceManager	12
2.2.6.2 NodeManager	12
2.2.7 MongoDB component	12
3 Implementation Details	13
3.1 Source code	13
3.1.1 Link to source code:	13
3.1.2 File Explanations:	13
3.2 Environment-specific configuration files	15
3.2.1 Local Development (Docker Compose)	15
3.2.2 Production Developement (Kubernetes)	15
3.2.2.1 ConfigMaps (Non-Sensitive Configuration):	15
3.2.2.2 Secrets (Sensitive Configuration)	16
3.3 Deployment strategy	16
4 Lesson learned	16
4.1 Lessons on Data Ingestion	16
4.1.1 Problem Description	16
4.1.2 Approaches Tried	16
4.1.3 Final Solution	17
4.1.4 Key Takeaways	17

4.2	Lessons on Data Processing with Spark	17
4.2.1	Problem Description	17
4.2.2	Approaches Tried	17
4.2.3	Final Solution	17
4.2.4	Key Takeaways	18
4.3	Lessons on Data Storage	18
4.3.1	Problem Description	18
4.3.2	Approaches Tried	18
4.3.3	Final solution:	18
4.3.4	Key takeaways:	18
4.4	Lessons on Monitoring & Debugging	18
4.4.1	Problem Description	18
4.4.2	Approaches Tried	18
4.4.3	Final Solution	19
4.4.4	Key Takeaways	19
4.5	Lessons on Security & Governance	19
4.5.1	Problem Description	19
4.5.2	Approaches Tried:	19
4.5.3	Final Solution	20
4.5.4	Key Takeaways	20
4.6	Lessons on Fault Tolerance	20
4.6.1	Problem Description	20
4.6.2	Approaches Tried	20
4.6.3	Final Solution	20
4.6.4	Key Takeaways	21
5	Conclusion and Future Work	21
5.1	Conclusion	21
5.2	Future Work	21

1 Problem Definition

1.1 Project Introduction

The project focuses on collecting real-time weather data from the OpenWeatherMap API for 20 major cities globally, including Hanoi, London, and New York, etc. The system processes this data to extract key meteorological metrics such as temperature, humidity, wind speed, and rainfall volume. Beyond simple collection, the pipeline calculates derived insights and aggregations, such as daily weather summaries and city-based averages. Our project topic highlights the distributed nature of environmental data, which requires a robust pipeline capable of handling continuous streams (***Speed Layer***) while simultaneously archiving historical records for batch analysis (***Batch Layer***). For learning usage, we also develop a ***simple Machine Learning job*** using ***MLlib***.

1.2 Data Introduction

The data source for this project is the **OpenWeatherMap API**. The data is collected as JSON and transformed into a structured format before being pushed to Kafka. The schema is enforced strictly in the Spark Consumer and Producer:

Name	Data Type	Explanation
city_name	String	The name of the city (e.g., “Hanoi”, “London”).
datetime	Timestamp	The specific time the weather data was calculated or received.
temperature	Double	The current temperature in Celsius.
humidity	Double	The humidity percentage.
wind_speed	Double	Wind speed in meter/sec.
cloud_cover	Double	Cloudiness percentage.
visibility	Double	Visibility distance in meters.
air_pressure	Double	Atmospheric pressure.
air_pressure_sea_level	Double	Atmospheric pressure on the sea level.
air_pressure_ground_level	Double	Atmospheric pressure on the ground level.
apparent_temperature	Double	The apparent temperature.
wind_direction	Double	Wind direction in degrees.
rain	Double	Rain volume for the last 1 hour (mm). Defaults to 0.0 if no rain.
country	String	The country code (e.g., “VN”, “US”).

Table 1: Schema definition for weather data

1.3 Problem suitability for big data

Weather data exhibits continuous fluctuations in temperature, humidity, and wind patterns, making it a streaming data problem. Efficient processing of such environmental metrics is critical for providing real-time monitoring and immediate alerts via the Speed Layer. Additionally, the accumulation of long-term meteorological logs falls under the batch category, allowing us to gain hands-on experience in implementing a complete Lambda Architecture that combine both real-time ingestion and historical data aggregation.

1.4 Scope and Limitations

Our project involves collecting, storing, and processing both historical and real-time meteorological data. It applies big data techniques to compute weather aggregations (e.g., daily rainfall, average temperature) and visualize environmental trends for 20 major cities

globally. The scope is limited to a specific subset of cities and a fixed set of weather parameters (temperature, humidity, wind speed) retrieved from the OpenWeatherMap API. The system currently operates in a local Kubernetes test environment (Minikube) with a reduced replication factor before broader production deployment.

2 Architecture and Design

2.1 Overall architecture

The system follows a hybrid Lambda architecture incorporating real-time streaming and batch processing:

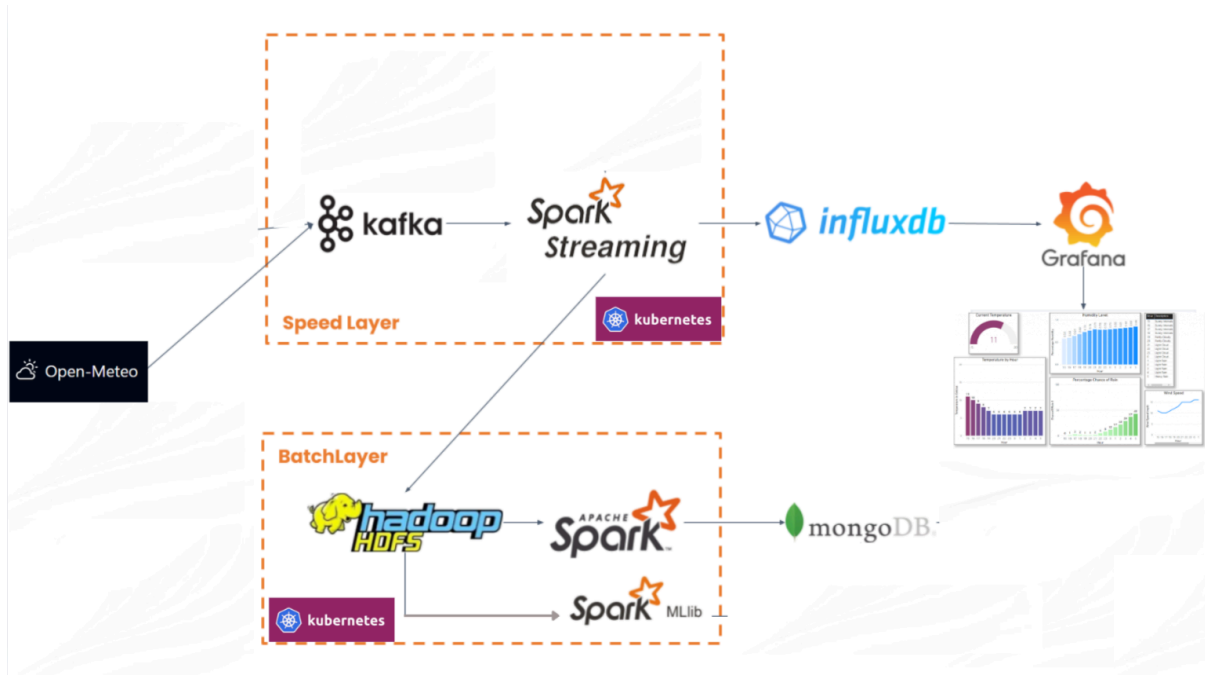


Figure 1: General architecture

2.2 Detail components

2.2.1 Kafka component

In terms of our Kafka implementation, our Kafka cluster is operated in **Kraft mode**, removing the Zookeeper dependency. This reduces the footprint and simplifies cluster management. The Kafka cluster contains:

- **1 controller pod:** Manage the overall Kafka cluster, including saving the metadata, connecting every brokers in the cluster.
- **3 brokers pods:** Store and streaming the data from **producer.py**.

For the deployment process, we utilize the Bitnami Kafka Helm chart. To ensure high availability and fault tolerance, the cluster is configured with a replication factor of three, ensuring data is mirrored across all brokers. Each topic is divided into **three partitions**, with partition leadership distributed evenly among the brokers. External access is enabled via the `_externalAccess_` parameter to facilitate connections from external clients, such as the Python producer. Consequently, the three brokers are exposed via local host ports 30092, 30093, and 30094. Regarding security, all communication protocols (CLIENT,

INTERNAL, EXTERNAL) are currently set to PLAINTEXT to resolve protocol mapping conflicts encountered during the development phase. However, for a production environment, we should implement SSL/TLS encryption to ensure secure communication.

In order to provide observability in Kafka for better visualization, we utilize deployed one more pod namely kafdrop. This pod provides the comprehensive view about Kafka cluster such as: status, data in each partition, broker, etc. As the kafdrop pod is not included in **Bitnami Kafka Helm chart**, we need to use external image from `obsidiandynamics/kafdrop`. The pod is configured with resource limits of 200m CPU and 256MB memory and is accessible via **localhost:30900**.

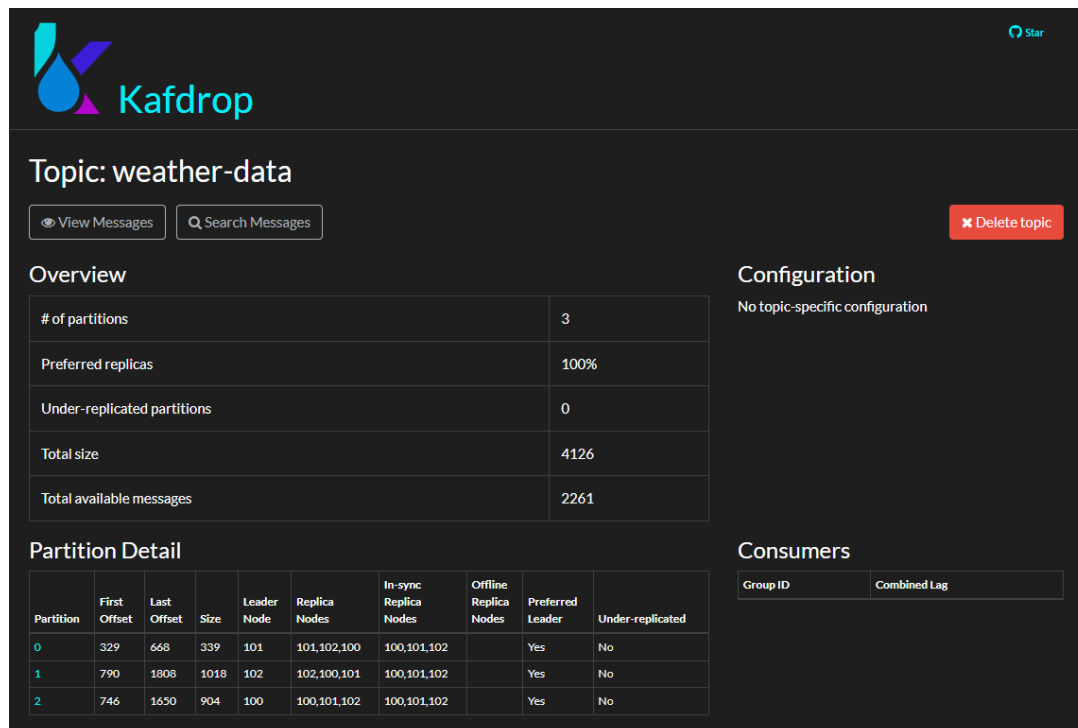


Figure 2: Kafdrop UI

2.2.2 Spark components

To ensure a consistent execution environment for both streaming and batch processing tasks, we utilize a custom Docker image named `spark-consumer`. This image is built upon the base `apache/spark:3.5.1` image and pre-installs necessary Python dependencies such as `influxdb_client`, `confluent-kafka`, `pandas`, and `numpy` to stream the data source.

Our Spark architecture is divided into two distinct deployments to serve the Lambda Architecture:

2.2.2.1 Spark Streaming Job (Speed Layer):

The real-time processing is handled by the `spark-streaming-consumer` Deployment. This pod is responsible for consuming data from the Kafka `weather-data` topic and implementing a “dual-sink” strategy using Structured Streaming’s `foreachBatch` function. This pod runs with the command `spark-submit --master local[*]`, effectively running the driver and executor in the same container. It dynamically loads the `org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1` package to handle Kafka integration. To prevent this inten-

sive streaming job from starving other cluster resources, we strictly define resource requests at **0.5 CPU / 1Gi Memory** and limits at **1 CPU / 2Gi Memory**. By running the `consumer.py` script, data is parsed against a strict schema and written simultaneously to **InfluxDB** (for real-time visualization) and **HDFS** (as raw Parquet files for historical storage). To maintain security and flexibility, all environment configurations are decoupled from the application code:

- **ConfigMaps:** Non-sensitive data such as `HDFS_OUTPUT_PATH` and `INFLUXDB_SERVER` are managed via `spark-consumer-config` and `spark-batch-config`.
- **Secrets:** Sensitive credentials, including the `INFLUXDB_TOKEN`, are securely injected into the pods using the `spark-consumer-secrets` Secret.

```
--- Processing Batch ID: 243 ---
Batch 243: Collected 1 records for InfluxDB.
Batch 243: Successfully wrote 1 points to InfluxDB.
Batch 243: Writing 1 records to HDFS at hdfs://hdfs-namenode.bigdata.svc.cluster.local:8020/weather-data...
Batch 243: Successfully wrote to HDFS.
--- Processing Batch ID: 244 ---
Batch 244: Collected 15 records for InfluxDB.
Batch 244: Successfully wrote 15 points to InfluxDB.
Batch 244: Writing 15 records to HDFS at hdfs://hdfs-namenode.bigdata.svc.cluster.local:8020/weather-data...
Batch 244: Successfully wrote to HDFS.
--- Processing Batch ID: 245 ---
Batch 245: Collected 4 records for InfluxDB.
Batch 245: Successfully wrote 4 points to InfluxDB.
Batch 245: Writing 4 records to HDFS at hdfs://hdfs-namenode.bigdata.svc.cluster.local:8020/weather-data...
Batch 245: Successfully wrote to HDFS.
--- Processing Batch ID: 246 ---
Batch 246: Collected 4 records for InfluxDB.
Batch 246: Successfully wrote 4 points to InfluxDB.
Batch 246: Writing 4 records to HDFS at hdfs://hdfs-namenode.bigdata.svc.cluster.local:8020/weather-data...
Batch 246: Successfully wrote to HDFS.
--- Processing Batch ID: 247 ---
Batch 247: Collected 16 records for InfluxDB.
Batch 247: Successfully wrote 16 points to InfluxDB.
Batch 247: Writing 16 records to HDFS at hdfs://hdfs-namenode.bigdata.svc.cluster.local:8020/weather-data...
Batch 247: Successfully wrote to HDFS.
```

Figure 3: Spark Streaming’s continuous status log

2.2.2.2 Spark Batch Job (Batch Layer)

For historical analysis, we deploy the `spark-batch` Deployment. Unlike the streaming job, this component simulates a scheduled task by running a shell loop that executes the **`write-to-mongodb.py`** script every 3 hours (sleeping for 18000 seconds between runs).

The **`write-to-mongodb.py`** job reads the raw Parquet files accumulated in HDFS, performs data de-duplication, and calculates aggregations such as average temperature, daily rainfall, etc for each city. In order to write the aggregated results into the `weather_db` database in MongoDB, it utilizes the `org.mongodb.spark:mongo-spark-connector_2.12:10.3.0` package with configured tokens to connect to MongoDB.

Phnom Penh	KH	29.117058823529415	30.84	24.48	63.11764705882353	1.68
Bangkok	TH	25.755333333333333	26.03	25.13	61.06666666666667	0.0
Da Nang	VN	24.423157894736843	25.0	23.99	78.0	0.58
Johannesburg	ZA	20.842142857142854	22.39	16.19	76.35714285714286	0.0
Madrid	ES	8.3	8.98	7.19	79.72222222222223	0.51
Nairobi	KE	21.716250000000002	23.79	20.02	60.0625	0.4200000000000004
Cairo	EG	19.472857142857134	21.33	18.33	56.42857142857143	0.0
Hanoi	VN	22.247999999999998	22.98	19.96	55.2	0.0
London	GB	10.17	10.59	7.74	84.5	0.0
Rome	IT	9.022307692307692	11.43	7.8	77.61538461538461	0.0
Berlin	DE	5.1940000000000002	5.62	3.66	90.53333333333333	0.11
Toronto	CA	-7.786428571428571	1.02	-10.0	79.21428571428571	0.0
Los Angeles	US	14.581875000000002	15.35	14.21	89.6875	0.0
New York	US	-5.521500000000001	3.77	-6.6	63.1	0.0
Buenos Aires	AR	18.257142857142863	30.82	16.94	73.57142857142857	0.0
Ho Chi Minh City	VN	29.850588235294122	31.69	25.57	71.82352941176471	0.0
Mexico City	MX	12.39588235294118	17.75	11.61	61.529411764705884	0.0
Paris	FR	5.532857142857144	8.34	5.11	95.0	0.0
Lagos	NG	28.543076923076924	29.94	27.12	79.92307692307692	0.0

city_name	country	date	daily_high_temp	daily_low_temp	daily_total_rain
Bangkok	TH	2025-12-03	25.61	25.13	0.0
Berlin	DE	2025-12-03	3.71	3.66	0.11
Buenos Aires	AR	2025-12-03	30.82	30.82	0.0
Cairo	EG	2025-12-03	21.33	21.33	0.0
Casablanca	MA	2025-12-03	15.78	15.78	0.0
Da Nang	VN	2025-12-03	25.0	25.0	0.0
Hanoi	VN	2025-12-03	19.98	19.96	0.0
Ho Chi Minh City	VN	2025-12-03	25.57	25.57	0.0
Johannesburg	ZA	2025-12-03	16.31	16.19	0.0
Lagos	NG	2025-12-03	29.12	29.12	0.0
London	GB	2025-12-03	7.76	7.74	0.0
Los Angeles	US	2025-12-03	15.35	15.2	0.0
Madrid	ES	2025-12-03	7.35	7.19	0.0
Mexico City	MX	2025-12-03	17.75	17.72	0.0
Nairobi	KE	2025-12-03	20.71	20.71	0.4200000000000004
New York	US	2025-12-03	3.77	3.72	0.0
Paris	FR	2025-12-03	8.34	8.34	0.0
Phnom Penh	KH	2025-12-03	24.93	24.48	1.68
Rome	IT	2025-12-03	11.43	11.34	0.0
Toronto	CA	2025-12-03	1.02	1.02	0.0

only showing top 20 rows

Writing batch views to MongoDB....
Write to MongoDB successfully
Finishing Batch Processing!
Sleeping for 5 minutes before next run...

Figure 4: Spark Batch's logs

2.2.2.3 Extra: Simple Linear Regression with MLlib

To have an insight about Spark MLlib, we develop a simple linear regression with our data for temperature prediction. We deploy a spark-ml-training Job that execute the job ml_training.py script to train the data. In our simple model, numerical features: **'humidity'**, **'wind_speed'**, **'cloud_cover'**, **'visibility'**, **'air_pressure'**, **'air_pressure_sea_level'**, **'air_pressure_ground_level'**, **'apparent_temperature'**, **'wind_direction'**, **'rain'** are chosen as independent variables to predict the **'temperature'** variables. The dataset is all the unique samples stored in HDFS. The process of our simple training progress is as follow (as stated in ml_training.py):

- Step 1 - **Reading Data:** Weather data is read from HDFS.
- Step 2 - **Data Cleaning & Feature Selection:** Remove duplicate records and extract relevant numeric columns to serve as features, while excluding non-predictive meta-data (like city_name, country) and the target variable (temperature).
- Step 3 - **Time-Based Splitting:** The data is sorted chronologically using the datetime column. The first 80% of the records are assigned to the Training Set, and the remaining 20% are assigned to the Testing Set (past data for training and future data for evaluation).

- Step 4 - **Normalization:** A `StandardScaler` is fitted on the training data only to normalize features (scaling to unit standard deviation and zero mean), and then applied to transform both training and testing sets. By using this way, we can avoid data leakage.
- Step 5 - **Model Training:** A Linear Regression model is initialized and trained on the prepared training data to predict the temperature.
- Step 6: **Evaluation:** The model generates predictions on the unseen test data. Performance is measured using RMSE (Root Mean Squared Error) and R^2 metrics.
- Step 7: **Model Saving:** The trained model is saved back to HDFS at `/models/weather_prediction_v1` for future use if needed.

By deploying a simple Machine Learning job, we are able to see how MLlib can be used in Big Data pipeline for future prediction. It should be noted that the current model is just for learning purpose; consequently, it lacks immediate real-world applicability and needs significant development to achieve notable performance.

2.2.3 InfluxDB component

InfluxDB serves as the dedicated Time-Series Database (TSDB) for the Speed Layer of our Lambda Architecture. It receives real-time weather data from the Spark Streaming consumer and provides the backing storage for live dashboards in Grafana. Unlike the Spark components, InfluxDB is deployed as a **StatefulSet** rather than a Deployment. This ensures a stable network identity and, most importantly, persistent storage binding. The container we use for this component is the official `influxdb:2.7` image. In order to prevent data loss during pod restarts or rescheduling, the pod is bound to a **PersistentVolumeClaim (PVC)** requesting **5Gi** of storage, mounted at folder `/var/lib/influxdb2` in our machine.

To streamline the deployment process, we leverage the container's native auto-initialization capabilities. By setting the environment variable `DOCKER_INFLUXDB_INIT_MODE` to `setup`, the instance automatically configures the initial organization, bucket, and administrative user upon the first launch. This eliminates the need for manual setup scripts or post-deployment CLI commands. Sensitive credentials are strictly managed through Kubernetes Secrets and are never exposed in the StatefulSet manifest. The `influxdb-secrets` object injects the following critical environment variables into the pod:

- `DOCKER_INFLUXDB_INIT_ORG`: Sets the default organization to `primary`.
- `DOCKER_INFLUXDB_INIT_BUCKET`: Creates the default bucket `primary` for storing weather data.
- `DOCKER_INFLUXDB_INIT_ADMIN_TOKEN`: Sets the API token required for authentication by both the Spark Consumer and Grafana.

Access to the database is restricted to internal cluster traffic to enhance security. We utilize a `ClusterIP` Service exposing port **8086**. In terms of this StatefulSet, no `NodePort` is exposed by default; external access for debugging is performed via `kubectl port-forward` when necessary.



Figure 5: InfluxDB UI

2.2.4 Grafana component

Grafana serves as the visualization platform for the real-time weather metrics stored in InfluxDB in our pipeline. It provides an accessible web interface for monitoring city-specific data trends such as temperature fluctuations, humidity levels, and rainfall. Grafana is deployed as a single-replica **Deployment** using the grafana/grafana-oss:10.3.1 image. To facilitate easy access from the host machine (browser), we expose the service via a **NodePort** on port **31300** (mapped to the container's internal port 3000). This allows users to access the dashboard directly at `http://localhost:31300` without complex ingress configurations. The configuration of Grafana to connect to MongoDB is set into two parts:

- **configmap-datasource.yaml**: defines the connection to InfluxDB using the internal cluster DNS `http://influxdb.bigdata.svc.cluster.local:8086`.
- **secret.yaml**: defines the username and password to log in into Grafana UI for dashboard visualization..

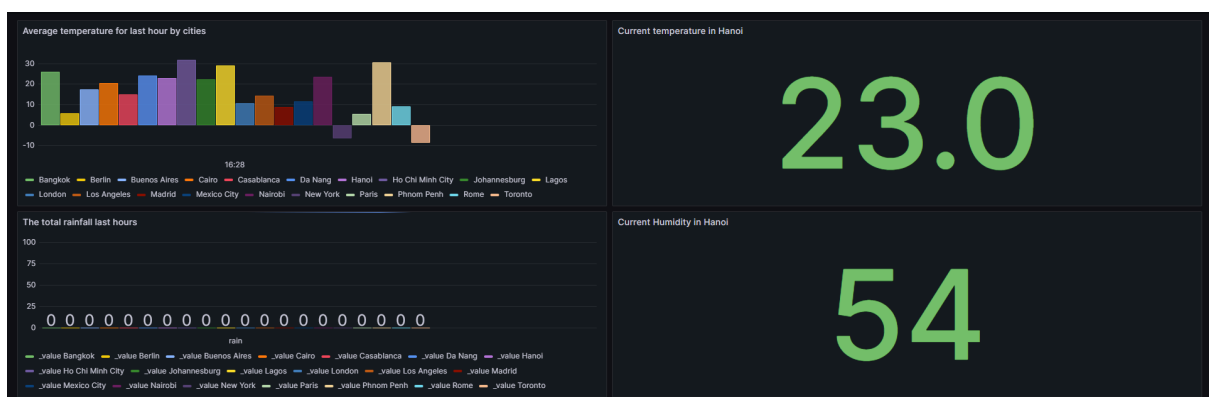


Figure 6: Grafana Dashboard

2.2.5 HDFS components

Hadoop Distributed File System (HDFS) serves as the “Batch Layer” storage in our architecture. It provides reliable, distributed storage for the raw Parquet files generated by the Spark Streaming consumer, which are subsequently processed by the Spark Batch job. Unlike stateless web services, HDFS components require stable network identities and persistent storage. Therefore, we deploy both the NameNode and DataNode as **StatefulSets** rather than Deployments.

2.2.5.1 NameNode

The NameNode manages the file system namespace and metadata of HDFS cluster. Before the main container starts, a shell script checks if the directory `/tmp/hadoop-root/dfs/name/current` is empty. If it is, it executes `hdfs namenode -format -force`. This ensures the filesystem is formatted automatically on the first launch but preserves data on subsequent restarts. It exposes port **8020** for RPC communication (used by DataNodes) and **9870** for the Web UI for better management.

2.2.5.2 DataNode

Two DataNode stores the actual data blocks in our HDFS cluster. It operates behind a headless service (ClusterIP: None) to allow direct peer-to-peer communication between nodes and clients. Each DataNode replica requests a **10GB** PVC mounted at `/tmp/hadoop-root/dfs/data`.

Summary

Security is off.

Safemode is off.

3,664 files and directories, 3,661 blocks (3,661 replicated blocks, 0 erasure coded block groups) = 7,325 total filesystem object(s).

Heap Memory used 206.05 MB of 980.5 MB Heap Memory. Max Heap Memory is 3.45 GB.

Non Heap Memory used 68.96 MB of 71.06 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	1.97 TB
Configured Remote Capacity:	0 B
DFS Used:	77.96 MB (0%)
Non DFS Used:	55.19 GB
DFS Remaining:	1.81 TB (92.17%)
Block Pool Used:	77.96 MB (0%)
DataNodes usages% (Min/Median/Max/stdDev):	0.00% / 0.00% / 0.00% / 0.00%
Live Nodes	2 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	2245
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Mon Dec 15 16:46:51 +0700 2025
Last Checkpoint Time	Mon Dec 15 16:47:06 +0700 2025
Enabled Erasure Coding Policies	RS-6-3-1024k

Figure 7: HDFS NameNode UI

2.2.6 YARN components

While our Spark jobs currently run in “client” mode (submitting tasks locally), we deployed the **YARN** (Yet Another Resource Negotiator) stack to complete the Hadoop ecosystem implementation, ensuring the cluster is ready for future scalable job scheduling. To ensure stability within a Docker/Kubernetes containerized environment, we tuned specific YARN configurations in the `hadoop-config` ConfigMap. We explicitly disabled physical and virtual memory checks to prevent YARN from killing containers during the development phase. YARN is also configured with the default queue to utilize 100% of the cluster resources for simplification. Overall, YARN consists of two notable pods:

2.2.6.1 ResourceManager

The ResourceManager is the master that arbitrates resources among all applications in the system. It is deployed as a standard Deployment using the `apache/hadoop:3` image. Port is also exposed at **8088** for the Cluster UI and **8032** for client job submissions.

2.2.6.2 NodeManager

The NodeManager runs on worker nodes to manage worker containers’s resource allocation. Unlike HDFS, YARN processing is transient. Therefore, the NodeManager uses an `emptyDir` volume for `/tmp/hadoop-yarn`, which is fast, ephemeral storage tied to the pod’s lifecycle.

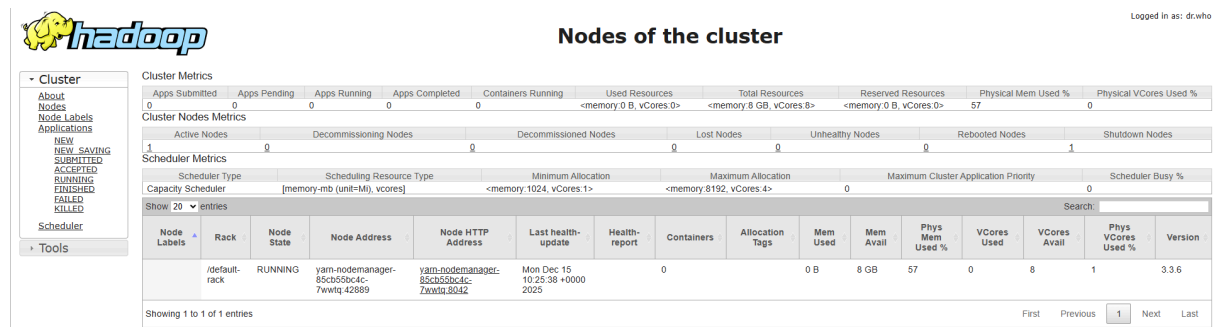


Figure 8: Yarn Resource Manager UI

2.2.7 MongoDB component

MongoDB acts as the “Serving Layer” for our Batch processing pipeline. While InfluxDB handles real-time time-series data, MongoDB is optimized for storing the structured, aggregated analytics results generated by the Spark Batch job, such as daily weather summaries and city-wide averages. Similar to our other stateful components, MongoDB is deployed as a **StatefulSet** to ensure stable network identity and storage persistence. For this component, we utilize the `mongo:7.0` image. A **5GB PersistentVolumeClaim (PVC)** is attached to the pod. This ensures that the valuable aggregated insights are preserved even if the database pod is restarted or rescheduled. For this development phase, authentication is disabled to simplify the integration with the `mongo-spark-connector`. However, the architecture supports enabling authentication via environment variables and Kubernetes Secrets for production. The database is structured to store specific batch views created by the batch processing job logic in database `weather_db`:

- **city_summary**: Stores statistical aggregates like average temperature, max/min temperature, and total rainfall per city.
- **daily_summary**: Stores daily historical records derived from the raw data.

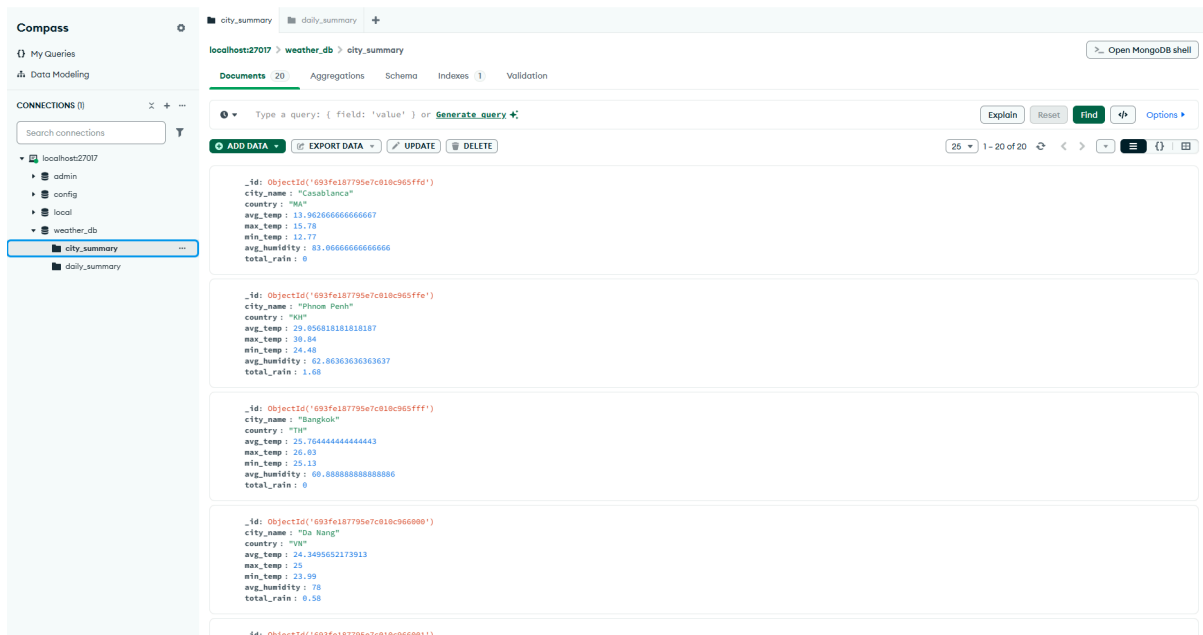


Figure 9: Batch view in MongoDB Compass

3 Implementation Details

3.1 Source code

3.1.1 Link to source code:

The link to our source code can be found here: [Github Source Code](#)

3.1.2 File Explanations:

The explanations for each file in our source code is as followed:

K8s Folder:

- Folder **kafka**:
 - **kafdrop-values.yaml**: configuration for *Kafdrop*'s *pod* such as image, brokerConnect, NodePort, etc.
 - **values.yaml**: overwrite configuration for release chart bitnami/kafka.
- Folder **spark**:
 - **batch-configmap.yaml**: stores configuration for the batch `write-to-mongodb.py` job, such as `HDFS_OUTPUT_PATH` and `MONGODB_URI`.
 - **batch-deployment.yaml**: defines the Deployment for the Spark batch job, which runs `write-to-mongodb.py` in a continuous loop.
 - **consumer-configmap.yaml**: stores non-sensitive configuration for the streaming consumer, including Kafka broker addresses and InfluxDB endpoints.
 - **consumer-deployment.yaml**: defines the Deployment for the Spark Streaming consumer, configured with specific resource limits and packages.
 - **consumer-secret.yaml**: stores sensitive credentials (base64 encoded) such as `INFLUXDB_TOKEN` for the `spark-submit-streaming pod`.

- **ml-job.yaml**: Define the machine learning **Job**. This pod is stated as **Job** type, as it runs once without automatically rerun.
- Folder **influxdb**:
 - **secret.yaml**: stores the initial administrator credentials, organization, bucket, and API tokens for InfluxDB setup.
 - **statefulset.yaml**: defines the StatefulSet for InfluxDB with persistent storage and the Service for internal cluster access.
- Folder **grafana**:
 - **configmap-datasource.yaml**: InfluxDB connection in Grafana using the secure token.
 - **deployment.yaml**: defines the Grafana Deployment, NodePort Service, and PersistentVolumeClaim for dashboard storage.
 - **secret.yaml**: stores the default admin credentials for the Grafana dashboard login.
- Folder **hdfs**:
 - **configmap.yaml**: defines Hadoop environment variables (e.g., YARN_SITE, MAPRED_SITE).
 - **hadoop-config-files.yaml**: stores the actual core-site.xml and hdfs-site.xml configuration files injected into the containers.
 - **hdfs-statefulsets.yaml**: defines the StatefulSets for both the HDFS NameNode (with formatting logic) and DataNode.
 - **services.yaml**: defines the Services for accessing HDFS (RPC/WebUI) and YARN components.
 - **yarn-deployments.yaml**: defines the Deployments for the YARN ResourceManager and NodeManager.
- Folder **mongodb**:
 - **statefulset.yaml**: defines the StatefulSet and Service for MongoDB 7.0 with persistent storage.

Other Files in Root Directory

- Folder **consumer**:
 - **consumer.py**: A Spark Structured Streaming script. It defines the strict schema (StructType) for weather data. It uses foreachBatch to handle the dual-sink logic (writing to InfluxDB and HDFS simultaneously) within the same micro-batch.
 - **write-to-mongodb.py**: The batch processing script. It reads Parquet data from HDFS, performs data cleaning (dropDuplicates), calculates aggregations (Mean, Max, Min), and writes the results to MongoDB using the mongo-spark-connector.
 - **ml-training.py**: The simple Linear Regression training script with data from HDFS. It fetches data from HDFS, train and save the model for further development.
- Folder **producer**:

- **producer.py**: The entry point for data ingestion. It initializes a producer with `enable.idempotence=True` for reliability. It spawns a separate thread for each of the 20 cities to ensure concurrent data fetching.
- **producer_utils_v2.py**: Contains the `retrieve_weather_data` function. It fetches data from the API, normalizes the timestamp, constructs a dictionary record, and sends it to Kafka using a callback for delivery reports.
- **producer_utils.py**: First version of our `retrieve_weather_data` function with less data.
- **docker-compose.yaml**: developed during our phase 2 (docker compose). This configuration file used for deploying and testing the full pipeline on a local Docker environment before migrating into Kubernetes.
- **spark.Dockerfile**: instruction file to build the custom Spark image with Python dependencies for processing jobs (`confluent-kafka`, `influxdb-client`, etc.).

3.2 Environment-specific configuration files

The application configuration relies on specific configuration strategies depending on the platform.

3.2.1 Local Development (Docker Compose)

During the initial development phase, we utilized a central `.env` file. This approach allows developers to easily modify parameters without altering the codebase or the `docker-compose.yaml` manifest.

- **Python Application**: The `producer.py` script utilizes the `dotenv` library to load environment variables from the `.env` file located in the project root. This includes variables like `KAFKA_EXTERNAL_SERVERS` and `OPENWEATHER_API_KEY` for fetching data from API to Kafka.
- **Infrastructure Config**: The `docker-compose.yaml` file references these variables to configure container environments. For example, the Kafka broker configuration relies on `${DOCKER_HOST_IP}` to set up advertised listeners correctly, and InfluxDB initialization relies on `${INFLUXDB_DB_USERNAME}` and `${INFLUXDB_DB_PASSWORD}`.

3.2.2 Production Development (Kubernetes)

When migrating to Kubernetes, we replaced the flat `.env` file with native Kubernetes resources to support granular management and enhanced security.

3.2.2.1 ConfigMaps (Non-Sensitive Configuration):

We use ConfigMaps to store plain-text configuration data, ensuring that changing a database URL or a file path does not require rebuilding container images.

- **Application Config**:
 - **spark-consumer-config** configMap: defines endpoints such as `INFLUXDB_SERVER` (`http://influxdb.bigdata.svc.cluster.local:8086`) and `HDFS_OUTPUT_PATH`.
- **Infrastructure Config**:
 - **hadoop-config** configMap: stores the actual XML content for `core-site.xml` and `hdfs-site.xml`, defining critical HDFS parameters.

- **grafana-datasources** configMap: stores configuration allowing Grafana to automatically connect to InfluxDB.

3.2.2.2 Secrets (Sensitive Configuration)

To protect sensitive credentials, we utilize Kubernetes Secrets. These values are Base64 encoded and injected into pods as environment variables at runtime.

- **spark-consumer-secrets**: stores InfluxDB token for spark streaming’s connection to InfluxDB.
- **influxdb-secrets**: stores username, password, organization, bucket and token with read and write permission in InfluxDB.
- **grafana-secrets**: stores username, password to log in to Grafana for visualization.

3.3 Deployment strategy

The project deployment strategy is listed as followed:

- Phase 1 - **Data Acquisition**: Developed Python scripts to ensure stable API connection and data formatting.
- Phase 2 - **Docker Compose**: Deployed the full stack using `docker-compose.yaml` to verify component connectivity (networking, volumes) in a controlled environment.
- Phase 3 - **Kubernetes Migration**:
 - Converted Docker Compose services to K8s environment.
 - Used **Helm** for managing the complex Kafka installation.
 - Deployed **StatefulSets** for stateful components (HDFS, InfluxDB, Mongo).
 - Deployed **Deployments** for stateless compute (Spark jobs).
 - Deployed a simple Linear Regression model using collected data.

4 Lesson learned

4.1 Lessons on Data Ingestion

4.1.1 Problem Description

The data ingestion layer use a Python script **producer.py** executed on a local host external to the cluster. This script is responsible for pushing weather data into our Kubernetes pipeline. During the initial deployment of **producer.py** and **other pods**, verifying successful data transmission throughout the pipeline is hard to our team. Network instability or configuration errors lead to silent failures, where pods appear as “ready” but fail to transmit data. Consequently, there is a need for **comprehensive, transparent logging within each pod/statefulset** to ensure pipeline observability.

4.1.2 Approaches Tried

- **Approach 1: No Callback to track logs**: We initially tried running the pipeline without any callback function to track the data streaming inside each deployment/statefulset. Although the pipeline maybe still works correctly without the callbacks, it provides zero visibility. Therefore, it prevent us from fixing and managing the pipeline during development process.

- **Approach 2: Callback functions applied in each pod:** The logs are printed obviously within each pod. It makes our team’s debugging much easier within the pipeline.

4.1.3 Final Solution

- **Final Solution:** The logs are printed clearly for each deployment/statefulset to track the status of data. For example, for the kafka’s broker management, we integrated the *confluent_kafka* library’s callback mechanism. Every time *send_to_kafka* is called, we pass the *delivery_report* function as an argument. This function is triggered once the broker sends. For the spark streaming, every time the data is written successfully or unsuccessfully to InfluxDB or HDFS; a notification log is printed.
- **Results:** By seeing the log, we can clearly observe if the pipeline is working well or not. For example, when running script *producer.py*, we saw “Message delivered to weather-data [0]”, then we can confirmed that the Kubernetes NodePort networking was correctly configured and the brokers were reachable.

4.1.4 Key Takeaways

The key takeaways that we obtain from this lesson is always implement delivery callbacks and logs for pods to handle errors occurred during the pipeline gracefully.

4.2 Lessons on Data Processing with Spark

4.2.1 Problem Description

Our architecture follows the Lambda Architecture pattern with two separate Spark applications to run concurrently: a streaming job (*consumer.py*) for the speed layer and a batch job (*write-to-mongodb.py*) for the batch layer. In the initial deployment in Kubernetes, we observed that whichever job was deployed first (usually the streaming consumer) would *monopolize the available cluster resources*. The Kubernetes scheduler would fail to schedule the second job (the batch processor), leading it in a Pending state. This will interrupts the architecture, as the batch layer could not run, preventing the generation of historical insights and MongoDB aggregations.

4.2.2 Approaches Tried

- **Approach 1: Default Configuration :** We initially submitted the Spark jobs without specifying explicit Kubernetes resource allocations, relying on Spark’s default behavior. Spark by default attempts to *utilize all available cores* on the node to maximize parallelism. In a shared resource environment like our development cluster, this behavior led to *resource starvation for other pods* such as *spark-submit-batch*.
- **Approach 2: Kubernetes Resource Allocation:** We decided to enforce strict resource allocation using Kubernetes requests and limits.

4.2.3 Final Solution

- **Detailed solution:** By modifying the Deployment manifests to include explicit resources allocation for Spark Streaming, it ensures the Kubernetes scheduler reserves only a specific part of the CPU for the streaming job, leaving room for the batch job. For the **Streaming Consumer**, we explicitly configured requests: *cpu: "500m"* and *memory: "1Gi"*. This is sufficient for reading from Kafka and writing to InfluxDB without consuming a full available resources (leaving room for *spark-submit-batch*).

- **Results:** After applying these changes, the terminal `kubectl get pods` showed both `spark-streaming-consumer` and `spark-batch` in the Running state simultaneously. The CPU usage **remained balanced**, and neither job crashed due to Out-Of-Memory (OOM) errors.

4.2.4 Key Takeaways

The key takeaway that we obtain from this lesson is that in a Spark Kubernetes environment, **resource management is critical**. Therefore, we should always define `resources.requests` and `resources.limits` for Spark driver and executor pods. This prevents a single application from accidentally causing freezing on the rest of the cluster.

4.3 Lessons on Data Storage

4.3.1 Problem Description

To ensure data durability and availability, we need to find a data storage mechanism that supports data replication and distribution. Therefore, we decided to implement HDFS with data replication.

4.3.2 Approaches Tried

- **Approach 1: HDFS with 1 datanode (no replication):** Without replication, the data used less memory as it doesn't need to replicate data. But when we intentionally shut down the datanode, *spark-batch* cannot read the data from HDFS. Therefore, it **couldn't provide the batch views to MongoDB**.
- **Approach 2: HDFS with 2 datanodes (two replications):** With data replication in two datanodes, when we shut down the 1 datanode, HDFS automatically transforms data access to another datanode. Therefore, *spark-batch* works normally without freezing the pipeline.

4.3.3 Final solution:

By using HDFS with 2 datanodes, we ensure the durability and availability of our data storage; giving our pipeline robustness.

4.3.4 Key takeaways:

The key takeaway that we obtain from this lesson is that HDFS is an ideal choice for data storage that needs durability and scalability. Moreover, we learned that we must always configure a replication factor of at least two to ensure that HDFS can handle node failures without disrupting the Spark processing pipeline.

4.4 Lessons on Monitoring & Debugging

4.4.1 Problem Description

Kubernetes has a complex infrastructure. Distinguishing between infrastructure failures, conflict configuration and application errors became difficult, resulting in slow feedback loops and wasted troubleshooting time. Therefore, it raises the need to deploy our end-to-end pipeline in a **more simplified environment (Docker)** to debug the configurations between images well, and then transform it into Kubernetes environment.

4.4.2 Approaches Tried

- **Approach 1: Kubernetes First Development:** We initially attempted to write the application code and deploy it immediately via Helm charts and manifests. However,

debugging was ***extremely complicated*** as we haven't had the experience with any Big Data project before. Therefore, initial pipeline deployment via ***Kubernetes*** seems to be not a good start for our project.

- **Approach 2: Docker First Development for Stabilization:** We decided to focus on building ***a fully end-to-end pipeline in Docker first***. We treated Docker Compose as a local “integration test” environment to remove network segmentation, DNS complexities and resolve conflict configurations between images. In parallel with that, our team tried to ***learn about basic concepts for Kubernetes*** such as: pods, deployments, statefulsets, services, etc. When the pipeline in Docker works correctly, we mitigate it into Kubernetes.

4.4.3 Final Solution

- **Final solution:** We ***stabilized the pipeline on Docker first***. We verified that `producer.py` could write to Kafka, `consumer.py` could write to HDFS and container defined in the `docker-compose.yaml` file worked well. Only after the `spark.Dockerfile`, Python scripts and all containers were proven to work in this environment, we proceeded to translate the pipeline into Kubernetes Deployment and StatefulSet manifests.
- **Results:** This strategy reduced the migration time significantly. When we finally deployed to K8s, we rarely face up with the configuration errors between containers. The errors encountered were strictly infrastructure-related in Kubernetes (e.g., PVC binding or NodePort configuration), which were easier to identified and fixed.

4.4.4 Key Takeaways

The key takeaway from this lesson is that when building a pipeline with high complexity, ***always start simple***. If our pipeline is debugged in a complex environment like K8s, it becomes impossible to identify the root cause of failures. Therefore, we should ***use Docker Compose as a “Staging” environment*** for logic, configuration verification before pushing to a Kubernetes cluster.

4.5 Lessons on Security & Governance

4.5.1 Problem Description

The project required managing sensitive credentials such as database passwords (InfluxDB, MongoDB), API tokens (OpenWeatherMap, InfluxDB admin token), and configuration endpoints across multiple components for production usage. Initially, during the Docker Compose development phase, all these variables were written in a single `.env` file for simplicity. When migrating to Kubernetes, relying on a simple text file or environment variables in plain text became a security risk. In a shared cluster environment, exposing credentials directly in deployment (YAML files) means anyone with read access to the repository or the cluster configuration could view sensitive keys. A lack of proper secret management could lead to unauthorized access to the InfluxDB data or the Kafka cluster.

4.5.2 Approaches Tried:

During the development process, we have tried two different approaches:

- **Approach 1: Plain Text Environment Variables (Docker development):** We initially considered directly embedding the environment variables into the `env` section of

the Kubernetes Deployment. This approach is easy to implement but highly insecure. Credentials would be visible to everyone.

- **Approach 2: Kubernetes Secrets (Base64 Encoded):** While migrating to K8s, We decided to utilize its Secret objects. This involves creating secret YAML files (e.g., *consumer-secret.yaml*, *secret.yaml*) where data is stored in Base64 encoded format. The pods then reference these secrets using `envFrom.secretRef` or `valueFrom.secretKeyRef` configuration. This adds complexity to develop but significantly improves security by encode sensitive data from the application configuration.

4.5.3 Final Solution

- **Final solution:** For the solution, we implemented a strict separation between sensitive and non-sensitive configurations (approach 2). Non-sensitive configuration (URLs, file paths, topic names) is stored in **ConfigMaps** (e.g., *spark-consumer-config*, *spark-batch-config*). Sensitive data (tokens, passwords) is encoded in **Secrets** (e.g., *influxdb-secrets*, *spark-consumer-secrets*).
- **Results:** The system successfully connects all components (Spark to InfluxDB, Grafana to InfluxDB) using the secure token.

4.5.4 Key Takeaways

The key takeaway that we obtain from this lesson is that Kubernetes Secrets provide a standard mechanism to inject sensitive data. Therefore, in production development, we will always separate configuration (ConfigMap) from credentials (Secret).

4.6 Lessons on Fault Tolerance

4.6.1 Problem Description

In our Big Data pipeline, component failures are inevitable (restart pipeline, accidentally delete a pod, etc). We needed to ensure that our “Speed Layer” storage (InfluxDB) would not lose data upon restarting and that our message broker (Kafka) could accept data even if a broker node failed. Without proper configuration, deleting a database pod or restarting the server results in total data loss. Similarly, if the Kafka cluster relies on a single broker, any stop from a broker will stop the entire streaming pipeline.

4.6.2 Approaches Tried

- **Approach 1: Kafka with 1 broker and no persistent volume in InfluxDB:** Initially, we consider using basic deployments for all components. It means that Kafka Cluster works with only 1 broker and InfluxDB has no persistent volume. Therefore, if a database pod restarts or a broker fails, it starts with a fresh filesystem, leading to data loss. This is unsuitable for our pipeline.
- **Approach 2: Kafka with 3 brokers and persistent volume in InfluxDB:** We adopted Kubernetes `StatefulSet` resources for InfluxDB and a multi-replica strategy for Kafka with 3 brokers. This ensures that pods maintain the fault tolerance and persistent storage.

4.6.3 Final Solution

- **Final solution:** For the solution, we implemented fault tolerance at both the storage and service levels:

- **InfluxDB:** We deployed InfluxDB as a **StatefulSet** with a **PersistentVolumeClaim (PVC)** template. This instructs Kubernetes to provision networked actual memory storage that exists independently of the pod's lifecycle.
- **Kafka:** We deployed the cluster with `replicaCount: 3` in the Helm values. This creates three distinct broker pods (`kafka-broker-0`, `1`, `2`) running in parallel.
- **Results:** We validated this by manually deleting pods using `kubectl delete pod:`
 - **InfluxDB Test:** When we restart the `influxdb` pod, we verified that the pod reconnected to the existing `data-influxdb-0` PVC, meaning all buckets and measurements remained intact.
 - **Kafka Test:** We terminated one broker pod and the cluster remained operational because the other two brokers continued running. For the partitions, Kafka controller chooses another pod to be the leader.

4.6.4 Key Takeaways

The key takeaway that we obtain from this lesson is always consider Fault Tolerance in our pipeline, as the system will always encounter some unexpected failure during operation.

5 Conclusion and Future Work

5.1 Conclusion

This project successfully demonstrated the deployment of a comprehensive Big Data pipeline on Kubernetes. We developed a Docker Compose setup and migrating to a K8s architecture using Helm charts and custom components. The system is capable of end-to-end data processing: ingesting weather data, processing it in real-time, archiving it to HDFS, and serving aggregated views via MongoDB and Grafana.

5.2 Future Work

In order to have more insight about Big Data concepts, we might want to increase the number of nodes in the cluster, ingest more different data sources, etc. We would also want to try with some different design patterns such as Kappa,... to compare between different architectures.

Acknowledgements:

This work is supported and supervised by Dr. Tran Viet Trung under the course of *Big Data Storage and Processing - IT4043E*. Also, our research and project are partially supported by other students at School of Information and Communication Technology, Hanoi University of Science and Technology.